# Advait Paranjpe

GitHub: github.com/AdvaitParanjpe
LinkedIn: linkedin.com/in/advait-paranjpe

Email: advait_paranjpe@outlook.com
Mobile: +1 (213) 284-9936
Website: advaitparanjpe.com

## Education

**University of California, Los Angeles** | *M.S. in Electrical and Computer Engineering* Sep 2025 – Dec 2026

- Coursework: CPU/GPU Architecture, Memory Systems, Cache Coherence, Interconnect Design.
- GPA: 3.7/4.0

**The University of Manchester** | *B.Eng. in Electrical and Electronic Engineering* Sep 2022 – Jul 2025

- Dissertation: "Cascaded PLL Microwave Synthesizer for 5G systems."
- **First Class Honours** – highest UK degree classification.

## Experience

**Jaguar Land Rover North America** | *Incoming Semiconductor Applications Engineering Intern* Jul 2026 – Sep 2026

**VAST Lab, UCLA** | *Student Researcher – GPU/FPGA Acceleration* Jan 2026 – Mar 2026

- Profiled ROCm attention kernels and memory bottlenecks to identify FPGA acceleration opportunities for Engram LLM inference.
- Modeled FPGA cache architectures, increasing projected hit rate from ∼5% at 256 entries to ∼48% with a 64k-entry, 4-way design.
- Translated workload profiling into FPGA tradeoffs across cache capacity, lookup throughput, locality, and on-chip memory usage.

**NESO / National Grid ESO** | *Software Engineering Intern (two summers)* Jul 2024 – Sep 2025

- Re-architected a 100k+ LOC simulation engine through vectorization and pipeline optimization, cutting runtime by 10x.
- Optimized 100M+ records through chunked, memory-efficient execution, cutting RAM from 32 GB to 16 GB and runtime by 11x.
- Reduced CPU context-switching overhead through parallel API batching and staged processing, increasing throughput by 24x for time-sensitive market telemetry.

## Projects

**AXI4-Stream Packet Router** | *RTL Design and UVM Verification (SystemVerilog, UVM)* Jun 2026

- Designed and verified a 2-input, 4-output AXI4-Stream packet router with store-and-forward buffers, round-robin arbitration, packet locking, backpressure, and malformed-packet dropping.
- Built conventional and UVM environments with reusable agents/BFMs, monitors, independent reference models, scoreboards, protocol checks, virtual sequences, and explicit scenario coverage.
- Passed 32 deterministic closure seeds across conventional and UVM regressions, covering all input–destination pairs, contention, stalls, resets, drop modes, counter wrap, and head-of-line blocking.

**Agentic RTL Security** | *Evidence-Gated SoC Bug Discovery (SystemVerilog, Python)* May 2026

- Created a mini-SoC with ten seeded RTL/security bugs spanning MMIO, privilege checks, state handling, and protocol behavior.
- Built deterministic directed tests, random fuzzing, clean-versus-bug simulation oracles, and reproducible failure artifacts; the baseline regression detected all 10 seeded bugs.
- Implemented trace generation, failure minimization, and evidence-backed reporting to compare conventional and LLM-assisted verification workflows.

**Sparrow-V Verification** | *RISC-V Differential and Random Verification (SystemVerilog, C, Python)* Mar 2026 – Jun 2026

- Developed differential testing against a reference model for RV32I execution, custom vector operations, retirement behavior, and scalar-vector command/completion semantics.
- Verified stalls, dependencies, redirects, subword accesses, misalignment traps, exceptions, backpressure, and precise retirement through randomized campaigns and focused regressions.
- Added assertions, cycle/retirement tracing, golden C/Python models, and a 500-seed confidence campaign for deterministic verification closure.

## Skills

- **Verification:** SystemVerilog testbenches, UVM, SVA, scoreboards, functional coverage, constrained-random testing, protocol checking, and regression automation.
- **Methods:** Differential testing, reference models, fuzzing, failure minimization, waveform triage, assertions, coverage-driven regression, and scenario analysis.
- **Tools and Languages:** SystemVerilog, Verilog, Python, C/C++, Verilator, Icarus Verilog, GTKWave, Vivado, Yosys, Git, Linux.